

Solar Monitor

Self-Hosted Solar Inverter Dashboard

Deployment & Operations Guide

GoodWe GW6000ES • dtitsolutions/solar-tracker

Version 1 — June 2026

Contents

1. Overview

Solar Monitor is a self-hosted dashboard that reads a solar inverter on your local network and shows live power flow, history, and per-inverter detail — with no dependency on the manufacturer's cloud. It runs as a small Docker stack on a machine on the same network as the inverter.

What you get: a live power-flow view (2D circles and a 3D house), a history chart, a multi-inverter fleet view, multi-user logins with roles, Prometheus metrics, and an in-app updater that pulls new versions from GitHub.

1.1 Architecture at a glance

The stack is composed of small services, each in its own container:

Service	Role	Port
web (Node)	Public web tier; serves the dashboard, streams live data (SSE), proxies the API	8080
app (Python)	Collector + API; one worker process per inverter; enforces auth	internal
config-db (MariaDB)	Configuration, users, inverters	internal (phpMyAdmin 8081)
data-db (MongoDB)	Time-series readings	internal (mongo-express 8082)
redis	Live data bus between collector and web tier	internal

Note on the inverter link: the collector talks to the inverter directly over the LAN. Older GoodWe WiFi/LAN modules use UDP port 8899; newer dongles (the kind the SEMS+ app connects to locally) use Modbus TCP on port 502. See Section 6.

2. Requirements

- A 64-bit Linux host (AlmaLinux/Rocky/RHEL or Debian/Ubuntu) on the same LAN/subnet as the inverter.
- Docker Engine with the Compose plugin (installed in Section 3).
- Roughly 2 GB RAM and 5 GB free disk for the stack and its databases.
- The inverter's IP address, and outbound internet only if you want the in-app updater.

3. Installing Docker

You need Docker Engine plus the Compose plugin (the `docker compose` subcommand). The bundled script does it for you on both RHEL-family and Debian/Ubuntu hosts.

3.1 The easy way — `install-docker.sh`

```
sudo ./scripts/install-docker.sh
```

The script: removes conflicting old packages, adds Docker's official repository, installs Engine + CLI + Compose + Buildx, enables the service at boot, adds your user to the docker group, and (on RHEL) opens the Docker bridge through firewall.

After it finishes: log out and back in (or run `newgrp docker`) so you can run Docker without sudo.

3.2 Manual — AlmaLinux / Rocky / RHEL

```
sudo dnf -y install dnf-plugins-core
sudo dnf config-manager --add-repo \
    https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"      # then log out/in
```

3.3 Manual — Ubuntu / Debian

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
    -o /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.asc] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli \
    containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

3.4 Verify

```
docker --version
docker compose version
docker run --rm hello-world
```

4. Deploying Solar Monitor

Place the project at `/opt/solar-tracker` (a git checkout of `dtitsolutions/solar-tracker`), then build and start it with the bundled script.

4.1 The easy way — build.sh

```
cd /opt/solar-tracker
./scripts/build.sh          # build + start (detached)
./scripts/build.sh --no-cache # force a clean rebuild
./scripts/build.sh --logs    # build, start, then follow logs
```

`build.sh` bakes the current git commit into the image (so the in-app updater knows what is deployed), builds the images, starts the stack, records the deployed version, and prints the dashboard URL.

4.2 Manual

```
cd /opt/solar-tracker
GIT_SHA="$(git rev-parse HEAD)" docker compose up -d --build
```

4.3 First login

Open `http://<server-ip>:8080` and sign in with **admin / admin**. Change the password immediately under Settings. Add your inverter under Settings, giving it a name and IP.

Companion tools: `http://<server-ip>:8081` (phpMyAdmin, config DB) and `http://<server-ip>:8082` (mongo-express, readings).

4.4 Everyday commands

```
docker compose ps          # service status
docker compose logs -f app  # collector logs
docker compose restart app  # restart the collector
docker compose down        # stop the stack
```

5. Connecting Your Inverter

GoodWe inverters expose a local interface through their WiFi/LAN dongle. There are two different protocols depending on the dongle generation, and using the wrong one is the most common reason for "unable to connect" even when the IP pings.

Dongle generation	Local protocol	Port	How to read it
Older WiFi/LAN kit	GoodWe UDP (AA55 / Modbus-RTU over UDP)	UDP 8899	Built-in goodwe driver (auto/family)
Newer dongle (SEMS+ local)	Modbus TCP	TCP 502	Modbus-TCP driver

Important: a `ping` or `nc -u` "success" does not prove the inverter is answering — UDP is connectionless, so `nc -u` reports "Connected" for any routable IP. Use the CLI (Section 7) to confirm what the dongle actually speaks.

5.1 Which one do I have?

Run the scanner against the inverter IP:

```
.venv/bin/python cli.py scan 192.168.10.51
```

- **TCP 502 open** → newer dongle, Modbus TCP. Confirm with `cli.py modbus <ip>` and read it with `cli.py tcp <ip>`.
- **UDP 8899 replies** → older module; the built-in goodwe driver works. If auto-detect fails, set `SM_GOODWE_FAMILY=ES` for a `GW####ES`.

This deployment (GW6000ES with a newer dongle) uses Modbus TCP on port 502. The `device-info` read returns the inverter's real serial, which confirms decoding end-to-end.

5.2 Useful environment knobs

Variable	Purpose	Example
<code>SM_GOODWE_FAMILY</code>	Skip UDP auto-detect; force a protocol family	ES
<code>SM_GOODWE_TIMEOUT</code>	Per-read timeout (seconds)	2
<code>SM_GITHUB_REPO</code>	Repo the updater checks	<code>dtitsolutions/solar-tracker</code>

Variable	Purpose	Example
SM_GITHUB_BRANCH	Branch the updater tracks	main
SM_GITHUB_TOKEN	Token for a private repo	github_pat_...

6. In-App Updates

Settings → Updates lets an admin check GitHub for a newer commit and apply it. Because a container cannot rebuild its own stack, the app drops a trigger file that a small host-side systemd watcher acts on.

6.1 One-time host setup

```
cd /opt/solar-tracker
sudo cp scripts/solar-tracker-update.service /etc/systemd/system/
sudo cp scripts/solar-tracker-update.path /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now solar-tracker-update.path
GIT_SHA="$(git rev-parse HEAD)" docker compose up -d --build
```

6.2 How it works

- **Check:** the app asks GitHub for the latest commit and compares it to the deployed SHA. If the server has no internet, the browser does the comparison instead.
- **Notify:** a banner appears when a newer commit exists.
- **Apply:** writes `.update/request`; the systemd path unit runs `scripts/update.sh` which pulls, rebuilds, restarts, and records the new version.

Private repo: set `SM_GITHUB_TOKEN` so the check works, and give the `/opt/solar-tracker` checkout working git credentials (SSH deploy key or a token in the remote URL) so the pull works.

7. The Inverter CLI (cli.py)

cli.py talks to one inverter directly — no Docker, DB or web stack — which makes it the fastest way to diagnose connectivity and confirm the protocol. Run it on the host in a virtualenv:

```
python3 -m venv .venv
.venv/bin/pip install goodwe
.venv/bin/python cli.py read 192.168.10.51
```

Command	What it does
read <ip>	Connect (UDP) and print live readings; --family ES to force, --all, --json
families <ip>	Try every UDP family; report which connects
probe <ip>	Raw UDP reachability (ES AA55 + ET Modbus frames); no goodwe needed
wifi	Find GoodWe modules via UDP 48899 broadcast — reveals the current IP
scan <ip>	Scan TCP 502/8899 + UDP 8899/48899 to identify the interface

Command	What it does
<code>modbus <ip></code>	Probe Modbus TCP :502 (newer dongles)
<code>tcp <ip></code>	Read a Modbus-TCP inverter: device info + live values (--debug dumps registers)

Tip: the inverter's local interface allows only one client at a time. Close the SEMS+ app (and stop the stack with `docker compose down`) before running the CLI.

8. Troubleshooting

8.1 "Unable to connect to inverter"

1. **Confirm the protocol.** Run `cli.py scan <ip>`. TCP 502 open = Modbus TCP; UDP 8899 reply = older module.
2. **Check the IP.** Run `cli.py wifi`. If the module answers at a different IP, DHCP moved it — use the new IP and set a reservation.
3. **Free the session.** Close SEMS+ and stop other pollers; only one local client is allowed.
4. **Container vs host.** If the CLI works on the host but the dashboard does not, it is Docker networking (see 8.2).
5. **Power-cycle.** If a previously-working module goes silent, reboot the inverter / reseal the dongle.

8.2 Container can't reach the LAN or internet (AlmaLinux)

On RHEL-family hosts, `firewalld` often blocks traffic forwarded from the Docker bridge to other LAN hosts and the internet. Open it:

```
sudo firewall-cmd --permanent --add-masquerade
sudo firewall-cmd --permanent --zone=trusted --add-interface=docker0
sudo firewall-cmd --reload
sudo systemctl restart docker
```

If the compose bridge is not `docker0`, find it with `ip -o link | grep br-` and add that interface to the trusted zone. As an alternative, run the collector on the host network: `docker compose -f docker-compose.yml -f docker-compose.hostnet.yml up -d`.

8.3 GitHub check fails

- **Private repo** → set `SM_GITHUB_TOKEN`.
- **No server internet** → the browser-side check still works for a public repo.

9. Reference

9.1 Ports

Port	Service
8080	Dashboard (web)
8081	phpMyAdmin (config DB)
8082	mongo-express (readings)

Port	Service
502 (TCP)	Inverter Modbus TCP (newer dongles)
8899 (UDP)	Inverter GoodWe protocol (older modules)
48899 (UDP)	GoodWe module discovery

9.2 Bundled scripts

Script	Purpose
scripts/install-docker.sh	Install Docker Engine + Compose (RHEL/Debian)
scripts/build.sh	Build + start the stack with the commit baked in
scripts/update.sh	Host updater: git pull + rebuild + restart
scripts/probe.py	In-container reachability probe
cli.py	Standalone single-inverter diagnostic/reader

9.3 Key file locations

- /opt/solar-tracker — deployment directory (git checkout)
- .env — site configuration (IPs, tokens, family hints)
- .update/ — updater trigger + deployed-version record

— End of guide —